VOLUME 09 OF 10

Graduate Game-Engine and Graphics Engineering

Master's-level specialization

Derive algorithms, reproduce research, optimize parallel systems and defend measured engineering choices.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 41 Graduate numerical methods and research practice

41.1 Reading research as claims, assumptions and evidence • 41.2 Conditioning, sparse systems and iterative solvers • 41.3 Constrained optimization, derivatives and optimality conditions • 41.4 Experimental design, uncertainty and reproducible graduate work

Module 42 Advanced parallel and GPU algorithms

42.1 Parallel reductions, scans and work-span analysis • 42.2 Tiling, data reuse and asynchronous pipelines • 42.3 Irregular workloads, wavefront scheduling and load balance • 42.4 Portable performance and integration-quality evaluation

Module 43 Advanced rendering and reconstruction

43.1 Monte Carlo light transport and multiple importance sampling • 43.2 Real-time ray tracing, dynamic acceleration and hybrid pipelines • 43.3 Spatiotemporal reuse, reservoirs and denoising • 43.4 Differentiable and neural rendering under engineering constraints

Module 44 Advanced engine systems and simulation research

44.1 Large-world scheduling, streaming and critical-path control • 44.2 Deformable simulation, variational models and solver tradeoffs • 44.3 Distributed game state, consistency and overload behavior • 44.4 Formal specifications, replay and reliability-oriented engine design

Module 45 Compilers, shader languages and engine specialization

45.1 Intermediate representations, SSA and dataflow analysis • 45.2 Alias analysis, vectorization and legal optimization • 45.3 Domain-specific languages, specialization and safe JIT execution • 45.4 Compiler/hardware co-design and graduate artifact review

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 41

Graduate numerical methods and research practice

Prerequisites Gates B, F and H; rigorous linear algebra, calculus, probability and systems competence.

Four lessons

41.1 Reading research as claims, assumptions and evidence

Extract an implementable argument rather than only a paper summary.

41.2 Conditioning, sparse systems and iterative solvers

Choose a solver from mathematical properties and error requirements.

41.3 Constrained optimization, derivatives and optimality conditions

Derive and verify an optimization method for an engineering objective.

41.4 Experimental design, uncertainty and reproducible graduate work

Make a performance or quality claim that survives independent scrutiny.

Module project

Produce a graduate numerical notebook with derivations, reference implementations, conditioning experiments and a preregistered evaluation plan. Complete substantial assigned problem sets; the short lesson quizzes are only formative checks.

Core references

R42 Stephen Boyd and Lieven Vandenberghe — Convex Optimization

R54 MIT OpenCourseWare — 18.085 Computational Science and Engineering I

R09 MIT OpenCourseWare — 18.05 Introduction to Probability and Statistics

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 41.1 • MODULE 41

Reading research as claims, assumptions and evidence

Learning goal Extract an implementable argument rather than only a paper summary.

A research paper makes claims under assumptions using a method and evidence. First identify the problem, baseline, objective and scope. Then reconstruct the central derivation or algorithm without looking at the authors' code. Track definitions, units and where approximations enter. A result can be valuable while narrower than its abstract initially suggests.

Build a claim-evidence table: each claim points to an equation, proof, experiment or stated limitation. Distinguish reproduction of reported results from an independent implementation and from a new extension. Read related work to identify the actual contribution. Select original papers and author artifacts where available, and record versions. A negative reproduction result needs diagnostic investigation before concluding the original claim is false.

WORKED EXAMPLE

A method reported faster at equal image error is not necessarily faster at equal sample count. Reproducing the wrong comparison target tests a different claim.

PRACTICE + PASS CRITERION

Analyze one paper associated with R23, R44 or R55. Pass: a two-page claim map, a reconstructed central algorithm, at least three assumptions and a feasible reproduction target are reviewed by another reader.

ASSIGNED STUDY

R23: Further Reading sections for a chosen rendering topic; R44 or R55: one named original paper; handbook research-reading template.

CHECK YOUR UNDERSTANDING

1. Why distinguish equal-error from equal-sample comparisons?

2. What makes a claim-evidence table useful?

3. Does a failed reproduction immediately disprove a paper?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 41.2 • MODULE 41

Conditioning, sparse systems and iterative solvers

Learning goal Choose a solver from mathematical properties and error requirements.

A linear system Ax=b has numerical behavior governed partly by conditioning: sensitivity of the solution to changes in data. Residual r=b-Ax measures equation mismatch, but a small residual need not imply a small forward error for an ill-conditioned system. Scaling and problem formulation matter before choosing a faster kernel.

Sparse storage exploits structure, while iterative methods use repeated matrix-vector operations. Conjugate gradient requires a symmetric positive-definite setting for its standard guarantees. Preconditioning changes the system seen by the iteration to improve convergence without changing the intended solution when correctly applied. State stopping criteria, norm, precision and maximum work. Test a known solution and increasingly difficult conditioning, not only one friendly matrix.

WORKED EXAMPLE

If A=diag(1,10^-6), its 2-norm condition number is 10^6. A small perturbation in the second equation can produce a much larger relative change in its associated solution component.

PRACTICE + PASS CRITERION

Implement a sparse matrix-vector product and conjugate gradient for a verified SPD system. Pass: derive the relevant assumptions, compare residual and forward error, and evaluate a simple preconditioner over several conditioning levels.

ASSIGNED STUDY

R54: linear systems, positive definiteness and numerical methods; R07: orthogonality and eigenvalues; R42: numerical optimization background.

CHECK YOUR UNDERSTANDING

1. What is the 2-norm condition number of diag(1,10^-6)?

2. Why can a small residual coexist with substantial solution error?

3. What matrix property is required by standard conjugate gradient?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 41.3 • MODULE 41

Constrained optimization, derivatives and optimality conditions

Learning goal Derive and verify an optimization method for an engineering objective.

Optimization requires a defined objective, feasible set and numerical tolerance. For differentiable unconstrained problems, stationary points have zero gradient but may be minima, maxima or saddles. Convexity provides stronger global conclusions under the appropriate assumptions. Equality and inequality constraints introduce Lagrange multipliers and Karush-Kuhn-Tucker conditions, whose necessity or sufficiency depends on regularity and convexity.

For nonlinear least squares, residual Jacobians support Gauss-Newton-style approximations; poor scaling or rank deficiency can require damping or regularization. Check analytic or automatic derivatives against finite differences over sensible step sizes. Gradient checks themselves have truncation and roundoff error. Report convergence histories and failed initializations rather than only a final successful parameter set.

WORKED EXAMPLE

Minimize x^2+y^2 subject to x+y=1. The Lagrangian gives 2x+lambda=0 and 2y+lambda=0; together with the constraint, x=y=0.5 and lambda=-1.

PRACTICE + PASS CRITERION

Derive the example, then solve a constrained fitting or IK problem with a numerical method. Pass: derivative checks, feasibility residual, convergence criteria and at least one failure or sensitivity case are reported.

ASSIGNED STUDY

R42: convexity, duality, optimality and numerical methods; R41: gradients and constrained differentials.

CHECK YOUR UNDERSTANDING

1. What is the constrained minimizer in the example?

2. Does zero gradient prove a general nonconvex point is a minimum?

3. Why vary finite-difference step size in a derivative check?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 41.4 • MODULE 41

Experimental design, uncertainty and reproducible graduate work

Learning goal Make a performance or quality claim that survives independent scrutiny.

Define the primary outcome and comparison before examining final results. Choose workloads that represent the intended claim, plus adverse and held-out cases. Repeated frames within one run can be correlated, so the statistical unit might be a run, scene or device rather than every frame. A large count of correlated observations does not automatically provide strong independent evidence.

Use confidence intervals or other justified uncertainty summaries, report effect sizes and preserve raw observations. Ablations remove or replace components to test which parts cause an improvement. Multiple comparisons and parameter tuning can overfit the benchmark suite. Keep final evaluation data separate from tuning when feasible. State external validity limits, including devices, workloads and quality settings not tested.

WORKED EXAMPLE

Ten thousand frames from one scene are not equivalent to ten thousand independent games. A scene-level comparison across diverse scenes can better test a claim about broad engine behavior.

PRACTICE + PASS CRITERION

Preregister a small experiment with baseline, primary metric, trial unit and exclusion policy. Pass: raw data and analysis scripts reproduce the tables, and an ablation isolates at least one proposed mechanism.

ASSIGNED STUDY

R09: statistical inference; R42: experimental examples; handbook benchmark and research templates.

CHECK YOUR UNDERSTANDING

1. Why identify the statistical unit?

2. What does an ablation test?

3. Why reserve held-out workloads?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 42

Advanced parallel and GPU algorithms

Prerequisites Modules 18, 37-38 and 41.

Four lessons

42.1 Parallel reductions, scans and work-span analysis

Prove the structure of a parallel primitive before tuning it.

42.2 Tiling, data reuse and asynchronous pipelines

Overlap movement and computation without creating races.

42.3 Irregular workloads, wavefront scheduling and load balance

Design around variable work rather than assuming identical threads.

42.4 Portable performance and integration-quality evaluation

Separate algorithmic benefit from one device-specific tuning win.

Module project

Implement a reusable GPU reduction, scan or compaction primitive and integrate it into an engine workload. Compare a simple reference, a competent library baseline when available and your implementation at equal correctness requirements.

Core references

R37 NVIDIA — CUDA Programming Guide

R43 Samuel Williams, Andrew Waterman and David Patterson — Roofline performance model, technical report

R48 MIT OpenCourseWare — 6.823 Computer System Architecture

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 42.1 • MODULE 42

Parallel reductions, scans and work-span analysis

Learning goal Prove the structure of a parallel primitive before tuning it.

A reduction combines many values with an operator; a scan produces prefix combinations. Associativity supports regrouping mathematically, but floating-point addition is not exactly associative. Decide whether the contract requires bitwise reproducibility, bounded error or an exact integer result. A tree reduction can reduce dependency depth from linear to logarithmic while retaining linear total work.

Work is the total number of operations; span is the longest dependency chain. More processors cannot beat the span without changing the algorithm. A naive parallel scan can have extra logarithmic work, while work-efficient constructions achieve linear work with logarithmic span under their model. Handle partial groups, empty inputs and irregular sizes explicitly. Cross-group coordination must follow the API rather than assuming a global barrier inside an ordinary dispatch.

WORKED EXAMPLE

A balanced reduction over 1,024 elements has 10 combination levels under a binary-tree model and 1,023 total pairwise combinations. Its implementation still pays memory and synchronization costs.

PRACTICE + PASS CRITERION

Implement reduction and either scan or compaction. Pass: derive work/span bounds, test sizes 0, 1 and non-powers of two, and report the numerical/reproducibility contract.

ASSIGNED STUDY

R37: cooperative execution and synchronization; R48: parallel architectures; derive algorithm bounds using Module 14 methods.

CHECK YOUR UNDERSTANDING

1. How many levels are in a balanced binary reduction of 1,024 values?

2. How many pairwise combines does that reduction require?

3. Why can parallel floating-point sums differ from serial sums?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 42.2 • MODULE 42

Tiling, data reuse and asynchronous pipelines

Learning goal Overlap movement and computation without creating races.

Tiling partitions a computation so values can be reused in a faster memory level. Matrix multiplication, image filters and neighborhood operations benefit when the tile fits available storage and exposes sufficient parallelism. Larger tiles can increase reuse but also raise register or shared-memory demand and reduce residency.

Double buffering can overlap preparation of one tile with computation on another. The pipeline needs explicit ownership: no consumer reads an incomplete tile and no producer overwrites data still in use. Asynchronous copy features vary by device; maintain a correct portable baseline. Account for boundary tiles, alignment, synchronization and extra staging work. Compare arithmetic intensity at the memory level actually being optimized.

WORKED EXAMPLE

For C=A*B, reusing a loaded A or B tile across many multiply-adds can raise useful operations per byte. A tile that spills heavily can erase that advantage.

PRACTICE + PASS CRITERION

Implement a tiled matrix multiply or image filter, then one overlap variant where supported. Pass: adversarial dimensions match the reference, synchronization is checked and the end-to-end comparison includes staging overhead.

ASSIGNED STUDY

R37: shared memory, synchronization and asynchronous operations where supported; R43: operational intensity.

CHECK YOUR UNDERSTANDING

1. Why can larger tiles hurt performance?

2. What ownership rule is essential for double buffering?

3. Why retain a non-asynchronous baseline?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 42.3 • MODULE 42

Irregular workloads, wavefront scheduling and load balance

Learning goal Design around variable work rather than assuming identical threads.

Ray traversal, graph processing and variable-length simulation tasks have irregular work. A fixed assignment can leave some workers idle while a few long tasks dominate the tail. Work queues, compaction and wavefront processing reorganize active tasks, trading scheduling and memory traffic for better coherence or load balance.

Persistent-worker designs keep workers alive to fetch tasks, but need bounded queues, termination detection and a memory-ordering contract. Global progress cannot depend on every workgroup being resident when the hardware does not guarantee that. Sorting tasks by similarity can improve coherence but has a cost and can alter latency. Evaluate both balanced and skewed distributions, including rare expensive tasks.

WORKED EXAMPLE

If 99 tasks take 1 unit and one takes 100 units, static one-task assignments can leave the final completion dominated by the long task. Splitting or scheduling work differently may help only when the task is divisible.

PRACTICE + PASS CRITERION

Implement an irregular workload using static assignment and a queue or compaction alternative. Pass: prove or test termination, handle queue capacity safely and measure tail behavior as well as mean throughput.

ASSIGNED STUDY

R37: execution, atomics and synchronization; R48: multithreaded scheduling; R23: wavefront rendering as an application.

CHECK YOUR UNDERSTANDING

1. Why can average task cost hide poor completion time?

2. Why avoid a global spin barrier requiring every group to be resident?

3. Is sorting tasks always beneficial?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 42.4 • MODULE 42

Portable performance and integration-quality evaluation

Learning goal Separate algorithmic benefit from one device-specific tuning win.

A performance-portable implementation preserves correctness and obtains useful efficiency across its stated target set. This does not mean identical code or identical speed everywhere. Capability dispatch can select specialized paths while keeping a tested fallback. Tuning parameters should be bounded and recorded rather than silently selected from the final test set.

Compare against a competent baseline and include small, large and irregular workloads. Report speed, memory, numerical error and integration cost. A kernel that is faster only after excluding transfer or conversion work should make that boundary explicit. Reproducible artifacts include build scripts, device capability reports, raw data and failure cases. Lack of access to multiple GPUs is a limitation to disclose, not a reason to invent portability evidence.

WORKED EXAMPLE

A tile size tuned on one GPU can reduce occupancy or exceed shared-memory limits on another. Capability checks and independent tuning prevent that choice from becoming an invalid universal default.

PRACTICE + PASS CRITERION

Integrate the selected primitive into the engine and evaluate at least two workload regimes. Pass: application impact is measured, unsupported features fall back safely and any untested device claims are explicitly excluded.

ASSIGNED STUDY

R37 and R21: capability contracts; R43: performance models; Module 41 experimental protocol.

CHECK YOUR UNDERSTANDING

1. Does portable performance require one fixed launch configuration?

2. What should be said when only one GPU was tested?

3. Why measure integration cost?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 43

Advanced rendering and reconstruction

Prerequisites Modules 09-10, 21-23 and 41-42.

Four lessons

43.1 Monte Carlo light transport and multiple importance sampling

Derive estimators with correct densities and measures.

43.2 Real-time ray tracing, dynamic acceleration and hybrid pipelines

Budget traversal, building and shading as one system.

43.3 Spatiotemporal reuse, reservoirs and denoising

Evaluate variance reduction together with bias, history validity and motion.

43.4 Differentiable and neural rendering under engineering constraints

Test learned or gradient-based methods against explicit baselines.

Module project

Reproduce a bounded rendering result and add one justified extension. Derive the estimator or reconstruction objective, compare equal-quality cost, include temporal failure cases and preserve source scenes and evaluation scripts.

Core references

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

R44 Benedikt Bitterli and coauthors — Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting

R47 Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola — Dive into Deep Learning

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 43.1 • MODULE 43

Monte Carlo light transport and multiple importance sampling

Learning goal Derive estimators with correct densities and measures.

The rendering equation expresses outgoing radiance as emission plus an integral of incident radiance, scattering and geometry factors. Monte Carlo estimation samples an integral: for samples from density p, average f(x)/p(x) where p is positive over the relevant nonzero support. Correct units and probability measures matter; area and solid-angle densities are not interchangeable without a Jacobian.

Importance sampling aims to reduce variance by matching useful structure. Multiple importance sampling combines strategies with weights that sum appropriately under the estimator. The balance heuristic uses relative weighted densities, but sample counts and support must be included correctly. Russian roulette can terminate paths without bias when surviving contributions receive the proper probability compensation. Finite-variance independent sampling gives the familiar inverse-square-root standard-error scaling; do not assume it for every estimator.

WORKED EXAMPLE

With one sample from each of two strategies and densities p1=2 and p2=6 at a point, the balance weight for strategy 1 is 2/(2+6)=0.25.

PRACTICE + PASS CRITERION

Implement a path tracer with explicit light sampling and MIS. Pass: derive the estimator, verify a simple known integral, compare variance at equal cost and test emission and grazing-angle edge cases.

ASSIGNED STUDY

R23: Monte Carlo Integration, Light Transport and sampling strategies. Work the associated mathematical exercises, not only the implementation.

CHECK YOUR UNDERSTANDING

1. What is the balance weight for strategy 1 in the example?

2. Why is p(x)>0 on relevant support important?

3. What sample-count increase ideally halves standard error for independent finite-variance samples?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 43.2 • MODULE 43

Real-time ray tracing, dynamic acceleration and hybrid pipelines

Learning goal Budget traversal, building and shading as one system.

Ray-tracing acceleration structures trade build cost, memory and query cost. Dynamic scenes may update or rebuild structures; the best policy depends on movement, geometry and traversal degradation. A refit can be cheaper initially but leave poor bounds after large changes. Hardware-assisted traversal still needs correct synchronization and resource lifetime.

Hybrid renderers combine rasterized visibility with ray-traced shadows, reflections or indirect lighting. Each path needs a consistent material and coordinate model. Screen-space or cached approximations require fallbacks where information is absent. Measure acceleration build/update, tracing, shading, denoising and synchronization separately, then report total frame impact. Compare equivalent scene motion and image quality rather than static screenshots alone.

WORKED EXAMPLE

A faster traversal variant that adds 4 ms of build work to save 1 ms of tracing is a regression for a scene rebuilt every frame, but might help a scene reused for many frames.

PRACTICE + PASS CRITERION

Add one ray-traced effect to the engine or build a software hybrid prototype. Pass: dynamic-scene tests include total update cost, missing-information fallbacks and resource synchronization evidence.

ASSIGNED STUDY

R23: intersection acceleration and light transport; R21, R24 or R25: supported ray-tracing interfaces for an optional hardware route.

CHECK YOUR UNDERSTANDING

1. Why can a cheap BVH refit become a poor long-term choice?

2. Is saving 1 ms of tracing worth 4 ms of per-frame rebuild cost?

3. Why compare animated scenes?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 43.3 • MODULE 43

Spatiotemporal reuse, reservoirs and denoising

Learning goal Evaluate variance reduction together with bias, history validity and motion.

Temporal and spatial reuse exploit related samples across frames or nearby locations. Reusing data changes the sampling and correlation structure, so the estimator and weights must be derived rather than improvised. Reservoir sampling can retain a compact representative sample under a defined weighted selection rule; its rendering use requires additional treatment of target distributions and reuse.

Denoising reconstructs a signal from noisy observations using assumptions about spatial, temporal or learned structure. It can suppress noise while introducing bias, blur or ghosting. Visibility changes and incorrect motion vectors invalidate history. Distinguish estimator bias from reconstruction bias and assess moving sequences at equal total cost. The original 2020 ReSTIR paper is a historical reproduction option, not a claim to represent the latest technique.

WORKED EXAMPLE

If a moving object reveals a background, old foreground samples are not valid background evidence merely because they occupy the same pixel coordinate. History acceptance must account for correspondence and visibility.

PRACTICE + PASS CRITERION

Reproduce a small weighted-reservoir example or temporal denoiser, then evaluate on motion and disocclusion sequences. Pass: the selection/weight rule is derived, bias limitations are stated and results include sequence-level failure cases.

ASSIGNED STUDY

R44: original ReSTIR paper and its estimator assumptions; R23: sampling theory; read only the selected bounded method before expanding scope.

CHECK YOUR UNDERSTANDING

1. Why cannot reused samples always be treated as new independent samples?

2. Why is same-pixel history not enough to establish correspondence?

3. What should be compared in a denoised rendering benchmark?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 43.4 • MODULE 43

Differentiable and neural rendering under engineering constraints

Learning goal Test learned or gradient-based methods against explicit baselines.

Differentiable rendering studies how outputs change with scene or model parameters. Visibility discontinuities, stochastic sampling and approximations complicate gradients. A useful derivative must match the actual objective and parameterization; automatic differentiation alone does not guarantee the right physical or statistical estimator. Validate on small scenes with finite differences and known cases.

Neural rendering or learned reconstruction can trade preprocessing, training and memory for runtime quality or speed. Training data coverage, scene specificity, temporal stability and inference hardware constrain deployment. Compare a non-learned baseline and include training or preprocessing cost where the claim requires it. Separate interpolation within familiar conditions from generalization to new scenes, lighting or motion.

WORKED EXAMPLE

A reconstruction network tested on frames from its own training sequence may perform well without generalizing to a new level. Holding out entire scenes tests a stronger claim.

PRACTICE + PASS CRITERION

Implement a small inverse-rendering optimization or learned reconstruction experiment. Pass: gradient or generalization checks, a non-learned baseline, compute/memory accounting and a distribution-shift failure analysis are included.

ASSIGNED STUDY

R47: automatic differentiation, optimization and generalization; R23: light transport and sampling foundations; select one original method with Module 41's paper-review protocol.

CHECK YOUR UNDERSTANDING

1. Does automatic differentiation guarantee the desired gradient estimator?

2. Why hold out entire scenes for some rendering evaluations?

3. When should training cost be included?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 44

Advanced engine systems and simulation research

Prerequisites Gates F-H; Modules 41-43.

Four lessons

44.1 Large-world scheduling, streaming and critical-path control

Design a bounded system under changing content and workload.

44.2 Deformable simulation, variational models and solver tradeoffs

Derive a physical or constraint model before comparing visual results.

44.3 Distributed game state, consistency and overload behavior

Specify which guarantees survive network delay and partial failure.

44.4 Formal specifications, replay and reliability-oriented engine design

Use precise invariants to reduce the space of subtle failures.

Module project

Conduct an engine-system experiment with a formal model or numerical argument, a reference implementation and stress-tested integration. Choose one main depth area: world streaming, simulation, distributed state or reliability.

Core references

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R26 Epic Games — Programming with C++ in Unreal Engine

R54 MIT OpenCourseWare — 18.085 Computational Science and Engineering I

R55 Matthias Mueller and collaborators — Author publication archive: position-based simulation and XPBD

R50 YosysHQ — SymbiYosys documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 44.1 • MODULE 44

Large-world scheduling, streaming and critical-path control

Learning goal Design a bounded system under changing content and workload.

Large-world engines coordinate visibility, simulation, streaming and resource residency under changing demand. A frame graph can make dependencies explicit, but hidden shared state or unpredictable tasks can still expand the critical path. Partition work by ownership and locality while controlling cross-partition communication.

World-coordinate precision can degrade far from the origin; origin rebasing, hierarchical coordinates or higher precision each affect physics, rendering and networking interfaces. Streaming priorities should reflect gameplay deadlines and resource availability, not only distance. Build a workload model with bursts, cancellation and scarce memory. Evaluate tail latency and quality degradation under stress rather than only steady travel through a preloaded scene.

WORKED EXAMPLE

A camera teleport invalidates a distance-based prefetch assumption and can create a sudden working-set change. The engine needs placeholders and a bounded recovery policy rather than an unlimited synchronous load.

PRACTICE + PASS CRITERION

Implement a bounded world-streaming scheduler or large-coordinate experiment. Pass: teleport and memory-pressure tests preserve correctness, peak resources are measured and the critical path is explained with a trace.

ASSIGNED STUDY

R26: world, task and streaming systems for comparison; R20: engine architecture; R54: numerical conditioning where coordinate precision is studied.

CHECK YOUR UNDERSTANDING

1. Why can a camera teleport defeat ordinary prefetching?

2. Why is origin rebasing an interface issue?

3. What distinguishes a bounded degradation policy?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 44.2 • MODULE 44

Deformable simulation, variational models and solver tradeoffs

Learning goal Derive a physical or constraint model before comparing visual results.

Deformable simulation represents degrees of freedom and a model of forces, energies or constraints. Spatial discretization, boundary conditions and time integration together define the numerical problem. Finite-element or mass-spring approaches approximate different structures; a visually similar result does not prove equivalent material behavior.

Implicit methods can improve stability for stiff systems while requiring nonlinear or linear solves and introducing other errors. Position-based and compliant-constraint methods offer alternative formulations with their own parameters and convergence behavior. Derive one simple energy or constraint, explain its units and study timestep/iteration sensitivity. Separate stability, accuracy and physical fidelity. A stable method can be inaccurate, and an accurate local solver can still use an inappropriate material model.

WORKED EXAMPLE

For spring energy E=0.5*k*(x-rest)^2, the force is -dE/dx=-k*(x-rest). This derivative supplies a check before extending to a multi-particle system.

PRACTICE + PASS CRITERION

Implement a small cloth, elastic bar or compliant-constraint model and compare two integration/solver choices. Pass: derive the core equations, check gradients, quantify constraint or energy error and reproduce one bounded result from the assigned research.

ASSIGNED STUDY

R54: equilibrium, variational and differential-equation methods; R55: 2016 XPBD and related position-based simulation course notes.

CHECK YOUR UNDERSTANDING

1. What force follows from the example spring energy?

2. Does numerical stability guarantee physical accuracy?

3. Why vary timestep and solver iterations independently?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 44.3 • MODULE 44

Distributed game state, consistency and overload behavior

Learning goal Specify which guarantees survive network delay and partial failure.

Distributed engine services divide state across processes or machines. Some operations require a single authoritative order, while others tolerate delayed or approximate views. Consistency is a property of a specified operation and model, not a single adjective that applies equally to every subsystem. During communication failure, availability and agreement requirements can conflict.

Partition ownership, migration and recovery need explicit protocols. A world handoff must not create two authorities granting the same irreversible outcome. Queues require bounds and backpressure; retry storms can magnify overload. Test fault sequences involving delayed messages, crashes and duplicate requests. Use local process simulation before public infrastructure, and never claim internet-scale behavior from a two-process happy-path demo.

WORKED EXAMPLE

If two servers both accept ownership of the same entity during a failed handoff, both might award a unique item. A versioned ownership or fencing mechanism must prevent stale authority from committing that effect.

PRACTICE + PASS CRITERION

Model a two-partition entity handoff with injected failure. Pass: a stated safety invariant holds, recovery assumptions are explicit and overload is bounded without unbounded retry amplification.

ASSIGNED STUDY

R17-R18: transport and transactions; R20: game-state architecture; use Module 41's claim and experiment discipline for the distributed model.

CHECK YOUR UNDERSTANDING

1. Why is duplicate authority dangerous?

2. What is a retry storm?

3. Can a small local demo establish internet-scale reliability?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 44.4 • MODULE 44

Formal specifications, replay and reliability-oriented engine design

Learning goal Use precise invariants to reduce the space of subtle failures.

A formal model abstracts a system to states and transitions so properties can be examined precisely. Safety states that something bad never happens; liveness states that something good eventually happens under assumptions. An engine resource manager can have safety invariants about ownership and liveness obligations about eventual completion or cancellation.

Model checking explores the chosen abstraction, not every detail of the production implementation. Connect model transitions to real code and tests. Deterministic replay, fault injection and property-based testing complement formal reasoning by exercising implemented behavior. Avoid vacuous proofs from impossible environment assumptions. A reliability claim should identify the property, model, bound and implementation evidence separately.

WORKED EXAMPLE

For an asset request, safety might require that a released owner never receives a callback touching its memory. Liveness might require that each accepted request eventually completes or cancels, assuming the worker continues to run.

PRACTICE + PASS CRITERION

Model a small engine lifecycle or queue protocol and connect it to implementation tests. Pass: at least one safety and one conditional liveness property are defined, a deliberate violation is found and abstraction limits are documented.

ASSIGNED STUDY

R50: safety, liveness and assumptions as a formal-methods reference; R11 and Module 18: ownership/concurrency contracts; R26: system comparison.

CHECK YOUR UNDERSTANDING

1. How do safety and liveness differ?

2. Why must liveness assumptions mention scheduling or worker progress?

3. Does proving an abstract model automatically prove all implementation code?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 45

Compilers, shader languages and engine specialization

Prerequisites Modules 15-16, 18, 37 and 41-42.

Four lessons

45.1 Intermediate representations, SSA and dataflow analysis

Represent program meaning so transformations can be justified.

45.2 Alias analysis, vectorization and legal optimization

Separate transformations that are fast from transformations that preserve meaning.

45.3 Domain-specific languages, specialization and safe JIT execution

Trade generality for predictable game-engine computation.

45.4 Compiler/hardware co-design and graduate artifact review

Connect a transformation to architecture and application evidence.

Module project

Gate I: deliver a research-quality reproduction and extension in rendering, simulation, GPU algorithms or compilation. This module also requires a small compiler optimization with semantic tests, generated-code inspection and end-to-end measurements.

Core references

R14 LLVM Project — Kaleidoscope and LLVM tutorials

R11 C++ Core Guidelines editors — C++ Core Guidelines

R37 NVIDIA — CUDA Programming Guide

R48 MIT OpenCourseWare — 6.823 Computer System Architecture

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 45.1 • MODULE 45

Intermediate representations, SSA and dataflow analysis

Learning goal Represent program meaning so transformations can be justified.

An intermediate representation makes control flow, values and operations explicit. Static single assignment gives each value one defining assignment; merge points use a representation such as phi nodes to select values from predecessor paths. SSA simplifies many analyses but does not by itself establish program correctness.

Dataflow analysis computes properties over a control-flow graph, often through iterative fixed points on a chosen domain. Liveness identifies values that may be used later; constant propagation tracks values known under the analysis. Soundness means the analysis does not exclude behavior relevant to its guarantee, while precision controls how much useful information it retains. Dominance and control-flow edges must remain consistent after transformations.

WORKED EXAMPLE

After an if/else assigning x=1 or x=2, a merge can represent x=phi(1 from then, 2 from else). Replacing it with 1 unconditionally would be wrong unless the else path is proven unreachable.

PRACTICE + PASS CRITERION

Extend the tiny compiler with a control-flow IR and one dataflow analysis. Pass: derive transfer functions and a fixed-point stopping condition, test loops and merges, and verify transformed programs against an interpreter.

ASSIGNED STUDY

R14: LLVM frontend, control flow and optimization tutorials; R48: compiler/architecture interaction.

CHECK YOUR UNDERSTANDING

1. What does SSA require of each value?

2. Why is the example phi node not always constant 1?

3. What is the role of a fixed point in dataflow analysis?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 45.2 • MODULE 45

Alias analysis, vectorization and legal optimization

Learning goal Separate transformations that are fast from transformations that preserve meaning.

An optimization must preserve the language's permitted observable behavior under its assumptions. Alias analysis determines which references may identify overlapping storage. Incorrectly assuming no aliasing can make loop reordering or vectorization wrong. Undefined behavior, overflow rules and floating-point options affect what transformations are legal, so record the compilation contract.

Vectorization groups iterations only when dependencies and semantics permit it. Reductions, exceptional values and rounding can require special handling. Register allocation maps values to a limited physical register set; spills introduce memory traffic. Inspect generated code and test adversarial inputs. A faster compiler output that silently changes required results is a correctness failure, not an optimization.

WORKED EXAMPLE

For a[i]=a[i-1]+1, iterations depend on previous results. Treating all iterations as independent changes behavior unless a valid alternative algorithm preserves the recurrence.

PRACTICE + PASS CRITERION

Implement or study a loop transformation with an explicit legality check. Pass: overlapping-memory and boundary cases are tested, compiler flags are recorded and speed claims exclude transformations that violate the required numerical contract.

ASSIGNED STUDY

R14: optimization and code generation; R11: aliasing, lifetime and arithmetic guidance; R37: vector/parallel execution consequences.

CHECK YOUR UNDERSTANDING

1. Why is alias analysis important for reordering loads and stores?

2. Are iterations of a[i]=a[i-1]+1 independent?

3. Why record floating-point compiler options?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 45.3 • MODULE 45

Domain-specific languages, specialization and safe JIT execution

Learning goal Trade generality for predictable game-engine computation.

A domain-specific language restricts expression to make authoring, validation or optimization easier. A material graph, particle formula or animation expression can expose structure that a general compiler cannot easily infer. Define types, allowed effects and resource limits before implementing code generation.

Specialization precomputes decisions known from material or scene parameters, potentially reducing runtime branches at the cost of more variants and compilation work. Just-in-time compilation introduces startup latency, code-cache management and execution-security concerns. Cache keys must include meaningful inputs and tool versions. Untrusted scripts or graphs need capability and complexity limits. A restricted interpreter remains a useful fallback and semantic reference.

WORKED EXAMPLE

Specializing four binary material options can create up to 2^4=16 variants before other parameters are considered. Uncontrolled variant growth can increase build time and runtime cache misses.

PRACTICE + PASS CRITERION

Build a typed expression or material DSL with constant folding and a bounded interpreter or JIT route. Pass: invalid graphs fail safely, variant counts are measured and compiled outputs match the reference.

ASSIGNED STUDY

R14: language implementation and JIT; R26: material and scripting concepts for comparison; Module 20: trust boundaries.

CHECK YOUR UNDERSTANDING

1. How many combinations do four independent binary options create?

2. Why limit the effects allowed in a DSL?

3. Why keep an interpreter alongside compiled execution?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 45.4 • MODULE 45

Compiler/hardware co-design and graduate artifact review

Learning goal Connect a transformation to architecture and application evidence.

Compiler decisions influence memory layout, instruction mix, register demand and parallel scheduling. Hardware features are useful only when software can express and exploit them efficiently. A co-design experiment changes one interface, representation or instruction opportunity and measures its complete effect. Avoid attributing a gain to hardware when it actually comes from a stronger software baseline.

Graduate review requires a defensible claim, mathematical or semantic argument, implementation and reproducible evaluation. Gate I can focus on one earlier graduate topic, but must include baselines, ablations, uncertainty and limitations. Publication acceptance is not required; credible independent evaluation is. Preserve unsuccessful experiments and distinguish a reproduced known result from the learner's actual extension.

WORKED EXAMPLE

A new vector instruction that saves arithmetic but increases packing and spills may lose in the full engine. The experiment must measure both the generated kernel and its application integration.

PRACTICE + PASS CRITERION

Complete Gate I and defend one compiler transformation's legality. Pass: an independent reader reruns the central experiment, the claimed extension is precisely identified and raw evidence supports the conclusions.

ASSIGNED STUDY

R14 and R48: compilation and architecture; R49: co-design methodology; Module 41: research evaluation protocol.

CHECK YOUR UNDERSTANDING

1. Why can a faster instruction fail to improve the application?

2. What distinguishes reproduction from extension?

3. Is publication acceptance mandatory for Gate I?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.